

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



NHẬP MÔN TRÍ TUỆ NHÂN TẠO (CO3061)

HOMEWORK 1 GAME MINESWEEPER

GIẢNG VIÊN: T.S. NGUYỄN QUỐC MINH

SINH VIÊN: 1. [Tên SV 1] - [MSSV]
2. [Tên SV 2] - [MSSV]
3. [Tên SV 3] - [MSSV]
4. [Tên SV 4] - [MSSV]
5. [Tên SV 5] - [MSSV]
6. Tran trung Hieu - 2211015

TP. HỒ CHÍ MINH, THÁNG 03/2026



Mục lục

Danh sách thành viên & Phân công công việc	2
1 Giới thiệu (Introduction)	3
2 Cơ sở lý thuyết và Bài toán (Problem Statement & DFS)	3
2.1 Thuật toán Tìm kiếm theo chiều sâu (Depth-First Search - DFS)	3
2.2 Áp dụng DFS vào bài toán mở ô (Flood Fill)	3
3 Thiết kế hệ thống (System Design)	4
3.1 Kiến trúc chương trình	4
3.1.1 Sơ đồ kiến trúc	4
3.1.2 Mô hình (Model)	4
3.1.3 Giao diện (View)	5
3.1.4 Bộ điều khiển (Controller)	5
3.2 Biểu đồ lớp (Class Diagram)	6
4 Thiết kế thuật toán (Algorithm Design)	7
4.1 Lưu đồ thuật toán tổng quát	7
4.2 Lưu đồ thuật toán mở ô (Reveal)	8
4.3 Lưu đồ thuật toán Loang (Flood Fill)	8
5 Hiện thực chương trình (Implementation)	9
5.1 Công cụ và Thư viện sử dụng	10
5.2 Hiện thực chương trình	11
5.2.1 Luồng chương trình chính	11
5.2.2 Menu chọn độ khó	11
5.2.3 Khởi tạo trò chơi theo độ khó	12
5.2.4 Đặt mìn ngẫu nhiên	12
5.2.5 Tính số mìn lân cận	12
5.2.6 Tạo giao diện bàn chơi (Grid Buttons)	13
5.2.7 Gán sự kiện chuột phải	13
5.2.8 Vòng lặp sự kiện và phân tích event	13
5.2.9 Gọi logic: đặt cờ và mở ô	14
5.2.10 Logic mở ô (Reveal) và trạng thái thua	14
5.2.11 Flood Fill khi ô trống (neighbors = 0)	15
5.2.12 Đặt/Gỡ cờ (Flag)	15
5.2.13 Kiểm tra điều kiện thắng	16
5.2.14 Cập nhật giao diện theo trạng thái board	16
5.2.15 Kết thúc trò chơi (Game Over / Victory)	16
6 Kết quả thực nghiệm (Experimental Results)	18
6.1 Môi trường cài đặt	18
6.2 Demo các trường hợp	18
7 Kết luận (Conclusion)	21
7.1 Kết quả đạt được	21
7.2 Hạn chế và Hướng phát triển	21



Danh sách thành viên & Phân công công việc

STT	Họ và tên	MSSV	Nhiệm vụ được giao	% Hoàn thành
1	Thoại	2313323	Tổng hợp báo cáo, viết phần Giới thiệu, Kết luận, định dạng tài liệu	100%
2	Trần Văn Mạnh	2212003	Phân tích thuật toán DFS, lý do chọn DFS cho bài toán	100%
3	Nguyễn Đức Thịnh	2313284	Vẽ biểu đồ lớp (Class Diagram), mô tả kiến trúc Module	100%
4	Vũ	[MSSV]	Vẽ chi tiết Lưu đồ thuật toán (Flowchart) cho các hàm xử lý chính (Reveal, Flood Fill)	100%
5	Lê Thành Bách	2210182	Hiện thực chương trình	100%
6	Tran trung Hieu	2211015	Viết kịch bản test, chụp hình minh chứng, hướng dẫn cài đặt	100%

Bảng 1: Bảng phân công công việc

1 Giới thiệu (Introduction)

Minesweeper (Dò mìn) là một trò chơi giải đố logic kinh điển dành cho một người chơi. Trò chơi trở nên phổ biến toàn cầu khi được tích hợp sẵn trong hệ điều hành Microsoft Windows.

Mục tiêu của trò chơi là mở tất cả các ô vuông trong một bảng lưới (grid) mà không được kích nổ bất kỳ quả mìn nào ẩn giấu bên dưới. Bảng trò chơi bao gồm các ô vuông, mỗi ô có thể chứa mìn hoặc không. Khi người chơi mở một ô:

- Nếu ô đó chứa mìn: Trò chơi kết thúc ngay lập tức (Thua cuộc).
- Nếu ô đó không chứa mìn: Ô sẽ hiển thị một con số (từ 1 đến 8) biểu thị số lượng mìn nằm trong 8 ô lân cận (ngang, dọc và chéo).
- Nếu ô đó không có mìn và cũng không có mìn nào xung quanh (số 0): Trò chơi sẽ tự động mở các ô trống lân cận (cơ chế loang).

Người chơi sử dụng các con số gợi ý để suy luận logic và cắm cờ (flag) vào các vị trí nghi ngờ có mìn. Trò chơi kết thúc thắng lợi khi tất cả các ô không chứa mìn đều đã được mở.

2 Cơ sở lý thuyết và Bài toán (Problem Statement & DFS)

2.1 Thuật toán Tìm kiếm theo chiều sâu (Depth-First Search - DFS)

Thuật toán Tìm kiếm theo chiều sâu (Depth-First Search - DFS) là một thuật toán duyệt đồ thị cơ bản, trong đó việc thăm các đỉnh được thực hiện theo nguyên tắc đi sâu hết mức có thể theo từng nhánh trước khi quay lui để tiếp tục khám phá các nhánh khác.

DFS có thể được cài đặt bằng hai phương pháp chính: sử dụng đệ quy hoặc sử dụng ngăn xếp (stack) để mô phỏng quá trình duyệt. Trong cách tiếp cận đệ quy, ngăn xếp lời gọi hàm của hệ thống được tận dụng để lưu trữ trạng thái duyệt, trong khi phương pháp lặp sử dụng một stack do người lập trình quản lý nhằm tránh nguy cơ tràn stack khi xử lý dữ liệu lớn.

Quy trình hoạt động cơ bản của DFS gồm các bước:

- Bắt đầu từ một đỉnh khởi đầu.
- Đánh dấu đỉnh hiện tại là đã thăm.
- Lần lượt duyệt các đỉnh kề chưa được thăm, ưu tiên đi sâu vào từng nhánh.
- Khi không còn đỉnh kề hợp lệ, quay lui để tiếp tục duyệt các nhánh khác.

Trong trường hợp tổng quát, độ phức tạp thời gian của DFS là $O(V + E)$, với V là số đỉnh và E là số cạnh của đồ thị. Độ phức tạp không gian phụ thuộc vào độ sâu lớn nhất của quá trình duyệt, trong trường hợp xấu nhất có thể đạt $O(V)$. Nhờ đặc tính này, DFS đặc biệt phù hợp cho các bài toán khám phá vùng liên thông, tìm đường đi, phát hiện chu trình và xử lý các cấu trúc dữ liệu dạng lưới.

2.2 Áp dụng DFS vào bài toán mở ô (Flood Fill)

Flood Fill là một kỹ thuật phổ biến nhằm xác định và mở rộng một vùng liên thông trong không gian hai chiều. Trong trò chơi Minesweeper, Flood Fill được sử dụng để tự động mở các ô trống liên tiếp khi người chơi nhấp vào một ô không có mìn xung quanh.

Thuật toán DFS được áp dụng để cài đặt Flood Fill bằng cách duyệt lần lượt các ô kề nhau theo nguyên tắc đi sâu. Khi một ô có số lượng mìn xung quanh bằng 0, thuật toán sẽ tiếp tục lan sang các ô lân cận, tạo thành một vùng mở rộng liên thông. Quá trình này được thực hiện cho đến khi không còn ô hợp lệ nào để tiếp tục.

Cụ thể, thuật toán sử dụng một ngăn xếp để lưu trữ các ô cần xử lý và một tập hợp *visited* nhằm tránh việc duyệt lặp lại. Tại mỗi bước, thuật toán lấy một ô từ stack, đánh dấu là đã mở, sau đó kiểm tra 8 ô lân cận. Nếu một ô kề chưa được mở, không có cờ và không chứa mìn, ô đó sẽ được mở và đưa tiếp vào stack trong trường hợp số mìn lân cận bằng 0.

Cách tiếp cận này giúp đảm bảo rằng mỗi ô chỉ được xử lý tối đa một lần, do đó độ phức tạp thời gian của thuật toán trong bối cảnh Minesweeper là $O(M \times N)$, với M và N lần lượt là số hàng và số cột của bảng. Đồng thời, bộ nhớ sử dụng cũng ở mức chấp nhận được, phù hợp với kích thước bảng trong thực tế.

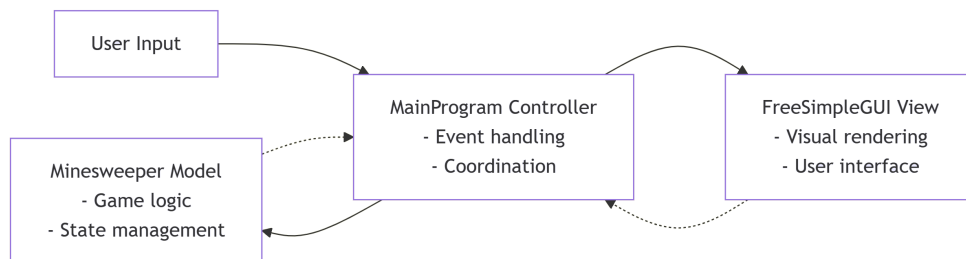
Việc sử dụng DFS cho Flood Fill mang lại nhiều ưu điểm như cài đặt đơn giản, hiệu quả về bộ nhớ, dễ kiểm soát luồng xử lý và đặc biệt phù hợp với đặc điểm vùng trống trong trò chơi Minesweeper, nơi không yêu cầu duyệt theo thứ tự khoảng cách.

3 Thiết kế hệ thống (System Design)

3.1 Kiến trúc chương trình

Trò chơi Minesweeper được xây dựng dựa trên mô hình kiến trúc **MVC** (Model-View-Controller), một mô hình phổ biến trong phát triển ứng dụng, đặc biệt là các ứng dụng có giao diện người dùng. Mô hình này giúp tách biệt ba thành phần chính: **Model** (dữ liệu và logic nghiệp vụ), **View** (giao diện người dùng) và **Controller** (điều khiển tương tác giữa người dùng, View và Model). Việc áp dụng MVC giúp mã nguồn dễ bảo trì, mở rộng và kiểm thử.

3.1.1 Sơ đồ kiến trúc



Hình 1: Kiến trúc MVC

3.1.2 Mô hình (Model)

Thành phần Model chịu trách nhiệm quản lý toàn bộ dữ liệu và logic nghiệp vụ của trò chơi, hoạt động độc lập với giao diện người dùng. Trong hệ thống, Model được cấu thành từ hai thực thể chính:

- **Cấu trúc dữ liệu Cell:** Đại diện cho một đơn vị ô trên bàn cờ. Mỗi ô lưu trữ trạng thái nội tại bao gồm:
 - **mine:** Xác định ô có chứa mìn hay không.
 - **open và flag:** Trạng thái hiển thị hiện tại (đã mở hoặc đang cấm cờ).
 - **neighbors:** Số lượng mìn nằm trong 8 ô lân cận (được tính toán trước khi bắt đầu màn chơi).
- **Lớp Logic Minesweeper:** Quản lý trạng thái tổng thể của ván chơi (`game_over`, `victory`) và lưới ô (`board`). Các phương thức xử lý chính bao gồm:
 - **Khởi tạo:** Thiết lập kích thước bảng và rải mìn ngẫu nhiên (`_place_mines`), sau đó tính toán chỉ số lân cận cho toàn bộ lưới.
 - **Xử lý tương tác:** Phương thức `reveal(r, c)` xử lý việc mở ô, kiểm tra điều kiện thua (nếu trúng mìn) hoặc kích hoạt thuật toán loang (`_flood_fill`) nếu mở trúng ô trống. Phương thức `toggle_flag(r, c)` quản lý việc đặt cờ đánh dấu.
 - **Kiểm soát trạng thái:** Phương thức `_check_victory()` liên tục kiểm tra điều kiện thắng sau mỗi lượt đi.

3.1.3 Giao diện (View)

Thành phần View đảm nhận vai trò hiển thị trực quan và tương tác với người dùng thông qua thư viện `FreeSimpleGUI`. View không chứa logic game mà chỉ phản ánh trạng thái dữ liệu từ Model.

- **Cấu trúc giao diện:** Hệ thống bao gồm Cửa sổ Menu (lựa chọn độ khó) và Cửa sổ Trò chơi (lưới các nút bấm và thông tin số mìn).
- **Cơ chế hiển thị:** Trạng thái của các nút bấm trên giao diện được ánh xạ trực tiếp từ thuộc tính của đối tượng `Cell` tương ứng:
 - **Trạng thái đóng:** Hiển thị màu xám hoặc ký tự cờ báo "P" (nếu `flag = True`).
 - **Trạng thái mở:** Hiển thị số lượng mìn lân cận, dấu "*" (nếu có mìn) hoặc để trống.
- **Cập nhật:** Giao diện được làm mới sau mỗi tương tác người dùng thông qua hàm `update_board`, đảm bảo tính đồng bộ giữa dữ liệu thực tế và hình ảnh hiển thị.

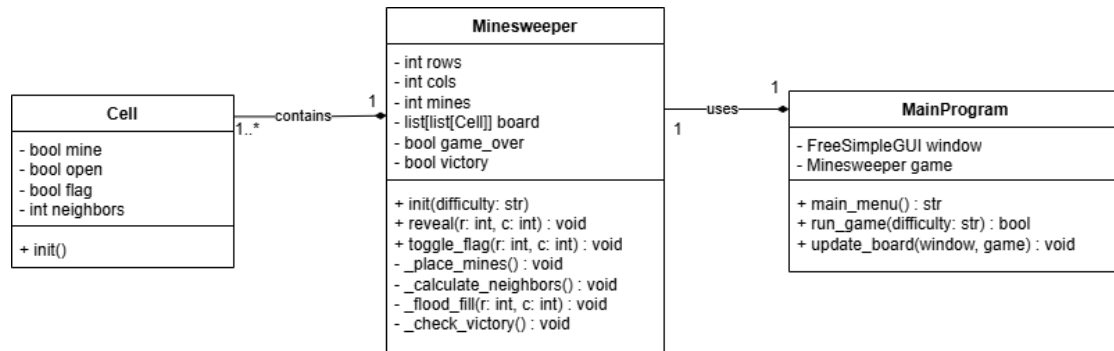
3.1.4 Bộ điều khiển (Controller)

Controller đóng vai trò trung gian điều phối luồng hoạt động của chương trình, được cài đặt chính trong tệp `main.py`.

- **Khởi tạo và điều hướng:** Hàm `main_menu()` tiếp nhận cấu hình độ khó từ người dùng, sau đó chuyển tiếp đến hàm `run_game(difficulty)` để khởi tạo đối tượng `Minesweeper` (Model) và cửa sổ game (View).
- **Vòng lặp sự kiện (Event Loop):** Controller liên tục lắng nghe các tín hiệu đầu vào từ thư viện GUI:
 - Chuột trái: Gọi `game.reveal()` để mở ô.
 - Chuột phải: Gọi `game.toggle_flag()` để cấm cờ.

- **Đồng bộ hóa:** Sau khi Model xử lý logic, Controller gọi phương thức `update_board()` để cập nhật lại View và kiểm tra các cờ hiệu `game_over` hoặc `victory` để đưa ra thông báo kết thúc trò chơi phù hợp.

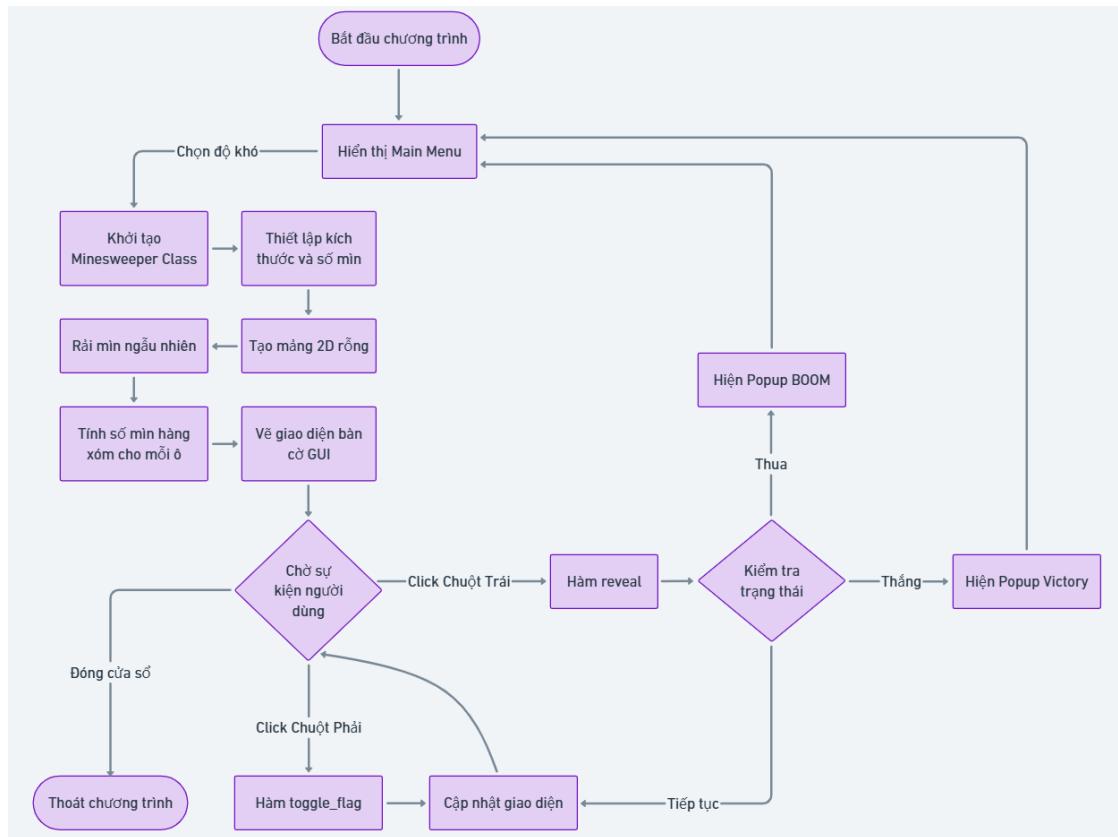
3.2 Biểu đồ lớp (Class Diagram)



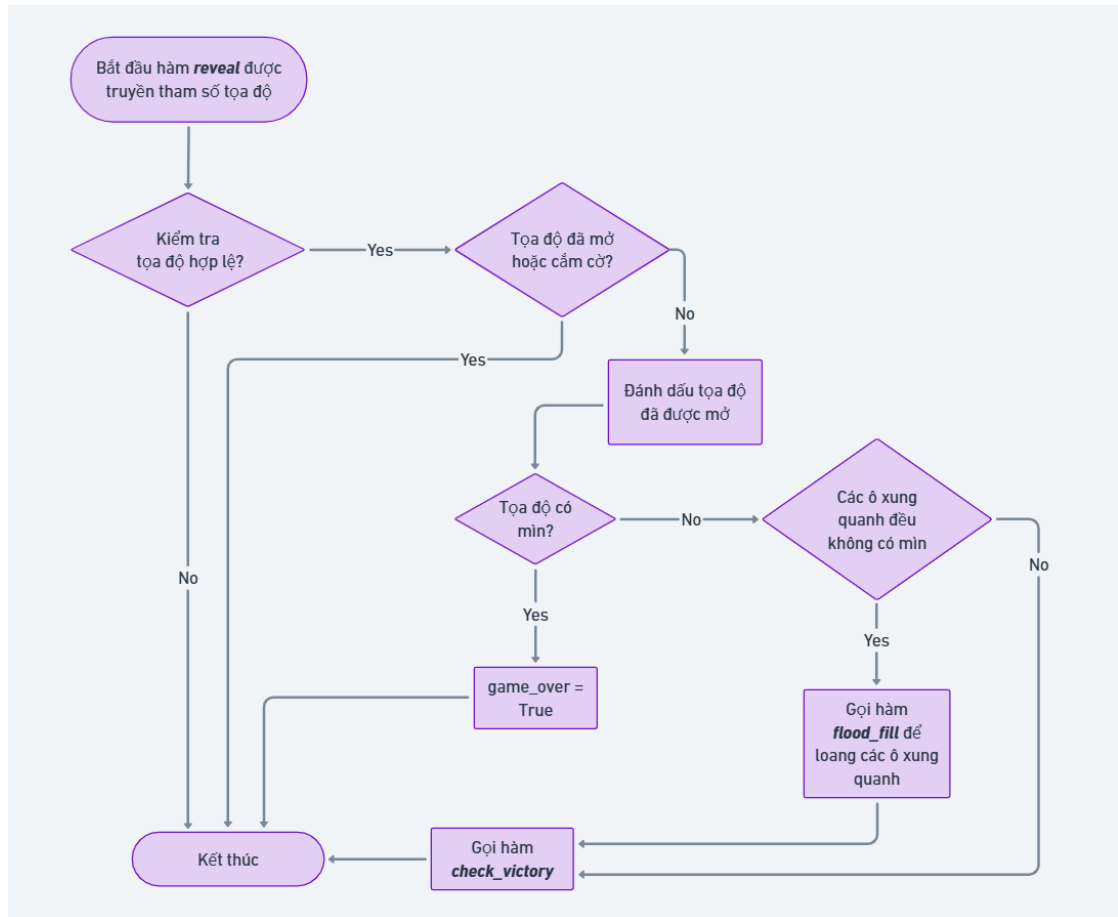
Hình 2: Biểu đồ lớp chương trình

4 Thiết kế thuật toán (Algorithm Design)

4.1 Lưu đồ thuật toán tổng quát

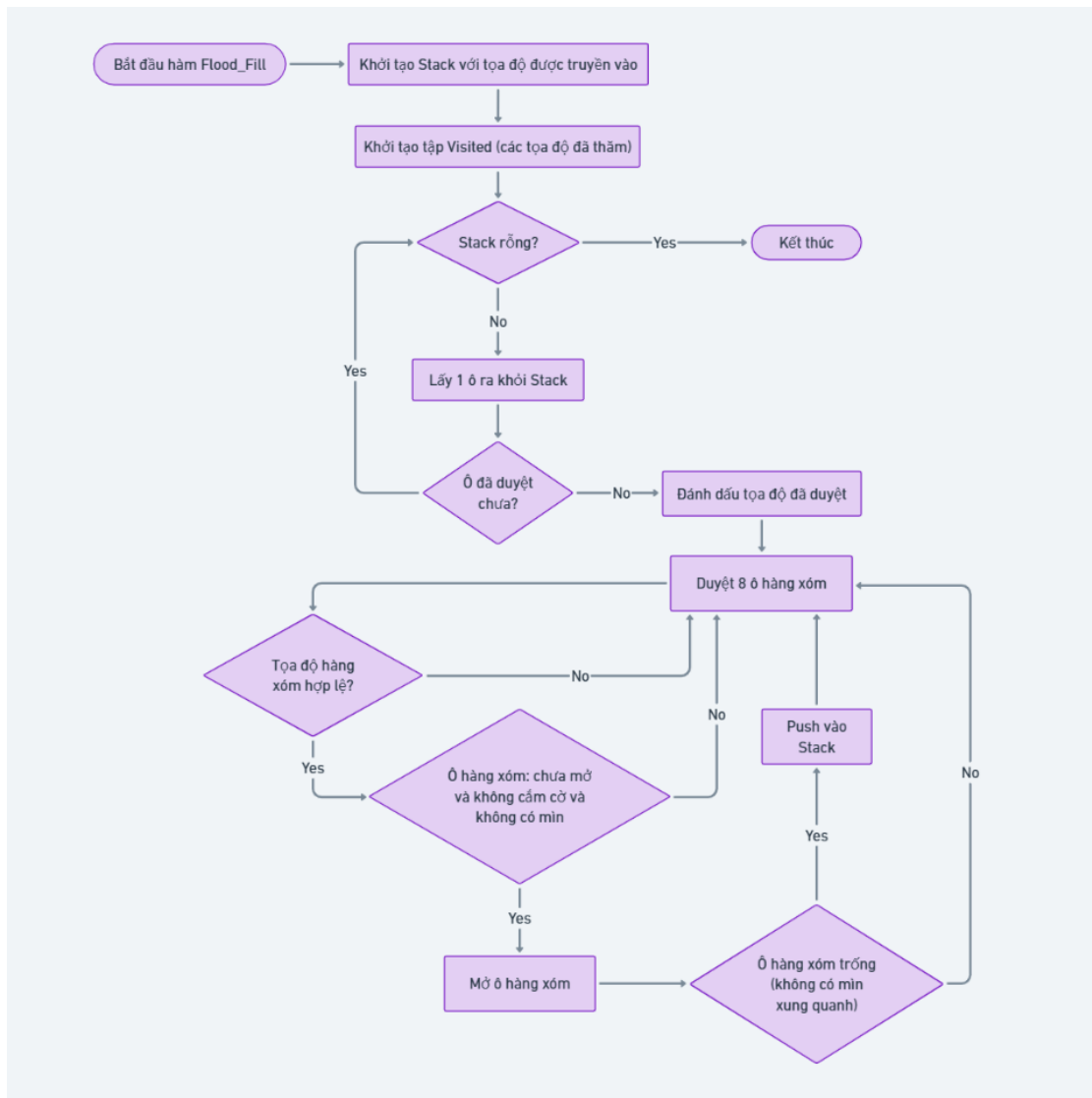


4.2 Lưu đồ thuật toán mở ô (Reveal)



4.3 Lưu đồ thuật toán Loang (Flood Fill)

Mục tiêu: mở lan các ô trống (không có mìn xung quanh) bằng giải thuật DFS.



5 Hiện thực chương trình (Implementation)

5.1 Công cụ và Thư viện sử dụng

Hệ thống trò chơi *Minesweeper* được cài đặt bằng ngôn ngữ lập trình **Python 3** (đã kiểm thử trên phiên bản Python 3.13), kết hợp với thư viện giao diện đồ họa **FreeSimpleGUI**. Việc lựa chọn các công cụ này nhằm đảm bảo tính đơn giản, dễ triển khai và khả năng mở rộng trong quá trình phát triển.

Các công cụ và thư viện chính bao gồm:

- **Python 3**: Ngôn ngữ lập trình chính, phù hợp cho việc xây dựng ứng dụng GUI nhờ cú pháp rõ ràng và hệ sinh thái thư viện phong phú.
- **FreeSimpleGUI**: Thư viện xây dựng giao diện người dùng dựa trên Tkinter, hỗ trợ xử lý sự kiện và bố cục giao diện một cách trực quan.
- **random**: Thư viện chuẩn của Python, được sử dụng để sinh vị trí ngẫu nhiên cho các mìn trên bàn chơi.

Về mặt kiến trúc, chương trình được chia thành hai thành phần chính:

- **Lớp Minesweeper**: Chịu trách nhiệm xử lý toàn bộ logic của trò chơi (đặt mìn, mở ô, kiểm tra thắng/thua).
- **Phần giao diện (GUI)**: Được cài đặt bằng thư viện FreeSimpleGUI, bao gồm giao diện menu chọn độ khó (hàm `main_menu`), giao diện bàn chơi chính với lưới các nút đại diện cho từng ô (hàm `run_game`) và cơ chế cập nhật trạng thái hiển thị dựa trên logic trò chơi (hàm `update_board`).

Cách tổ chức này giúp tách biệt rõ ràng giữa *logic xử lý* và *giao diện*, từ đó nâng cao khả năng bảo trì và mở rộng hệ thống.

5.2 Hiện thực chương trình

Chương trình được tổ chức theo hướng tách biệt giữa **giao diện (GUI)** và **logic trò chơi**. Phần GUI chịu trách nhiệm hiển thị, nhận thao tác người dùng; phần logic nằm trong lớp Minesweeper để xử lý luật chơi và trạng thái.

5.2.1 Luồng chương trình chính

```
1 if __name__ == "__main__":
2     while True:
3         dif = main_menu()
4         if not dif:
5             break
6         if not run_game(dif):
7             break
```

Listing 1: Vòng lặp chạy chương trình chính

Chương trình chạy trong vòng lặp vô hạn. Mỗi lần lặp sẽ: (1) mở menu chọn độ khó (`main_menu()`), (2) nếu người dùng thoát menu thì dừng, (3) nếu chọn chơi thì chạy game với độ khó đó (`run_game(dif)`). Nếu người dùng đóng cửa sổ game thì `run_game` trả về `False` và chương trình kết thúc.

5.2.2 Menu chọn độ khó

```
1 def main_menu():
2     sg.theme('DarkBlue12')
3     layout = [
4         [sg.Text("MINESWEEPER", font=("Helvetica", 24), justification='center',
5             expand_x=True)],
6         [sg.Frame("Difficulty", [
7             [sg.Radio("Easy (8x8)", "DIF", key='easy', default=True)],
8             [sg.Radio("Intermediate (16x16)", "DIF", key='intermediate')],
9             [sg.Radio("Expert (30x16)", "DIF", key='expert')]
10        ])],
11         [sg.Button("PLAY", size=(10, 2), bind_return_key=True), sg.Button("EXIT",
12             size=(10, 2))]
13     ]
14     window = sg.Window("Menu", layout, element_justification='c')
15     event, values = window.read()
16     window.close()
```

Listing 2: Tạo giao diện menu chọn độ khó

Hàm `main_menu` tạo cửa sổ menu gồm tiêu đề, nhóm radio chọn độ khó và hai nút `PLAY/EXIT`. Lệnh `window.read()` chờ người dùng thao tác và trả về: `event` (tên nút được nhấn) và `values` (trạng thái các radio).

```
1 if event == "PLAY":
2     if values['easy']: return 'easy'
3     elif values['intermediate']: return 'intermediate'
4     elif values['expert']: return 'expert'
5 return None
```

Listing 3: Trả về mức độ khó được chọn

Nếu người dùng nhấn `PLAY`, chương trình đọc radio được chọn để trả về chuỗi độ khó. Nếu không nhấn `PLAY` (ví dụ nhấn `EXIT` hoặc đóng cửa sổ), hàm trả về `None` để thoát.

5.2.3 Khởi tạo trò chơi theo độ khó

```
1 class Minesweeper:
2     def __init__(self, difficulty):
3         if difficulty == 'easy':
4             self.rows, self.cols, self.mines = 8, 8, 10
5         elif difficulty == 'intermediate':
6             self.rows, self.cols, self.mines = 16, 16, 40
7         else:
8             self.rows, self.cols, self.mines = 16, 30, 99
```

Listing 4: Thiết lập kích thước và số mìn theo độ khó

Khi khởi tạo `Minesweeper(difficulty)`, chương trình gán kích thước bàn chơi và số lượng mìn tương ứng. Độ khó `expert` dùng bàn 16x30 với 99 mìn.

```
1 self.board = [[{'mine': False, 'open': False, 'flag': False, 'neighbors': 0}
2                 for _ in range(self.cols)] for _ in range(self.rows)]
3 self.game_over = False
4 self.victory = False
5 self._place_mines()
6 self._calculate_neighbors()
```

Listing 5: Khởi tạo cấu trúc dữ liệu bàn cờ

Bàn cờ được lưu bằng ma trận 2 chiều. Mỗi ô là một dictionary gồm: `mine` (có mìn), `open` (đã mở), `flag` (đặt cờ), `neighbors` (số mìn kề). Sau đó chương trình đặt mìn và tính số mìn lân cận cho từng ô.

5.2.4 Đặt mìn ngẫu nhiên

```
1 def _place_mines(self):
2     count = 0
3     while count < self.mines:
4         r = random.randint(0, self.rows - 1)
5         c = random.randint(0, self.cols - 1)
6         if not self.board[r][c]['mine']:
7             self.board[r][c]['mine'] = True
8             count += 1
```

Listing 6: Đặt mìn ngẫu nhiên lên bàn cờ

Chương trình dùng vòng lặp đến khi đủ số mìn. Mỗi lần chọn ngẫu nhiên tọa độ (r, c) . Nếu ô chưa có mìn, đặt mìn và tăng biến đếm. Cách này đảm bảo không đặt trùng mìn trên cùng một ô.

5.2.5 Tính số mìn lân cận

```
1 def _calculate_neighbors(self):
2     directions = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
3     for r in range(self.rows):
4         for c in range(self.cols):
5             if self.board[r][c]['mine']:
6                 continue
7             count = 0
8             for dr, dc in directions:
9                 nr, nc = r + dr, c + dc
10                if 0 <= nr < self.rows and 0 <= nc < self.cols:
11                    if self.board[nr][nc]['mine']:
```

```
12         count += 1
13         self.board[r][c]['neighbors'] = count
```

Listing 7: Tính số mìn kề của từng ô

Với mỗi ô không phải mìn, chương trình duyệt 8 hướng xung quanh để đếm số ô có mìn. Kết quả được lưu vào trường `neighbors`, dùng để hiển thị số trên giao diện khi người chơi mở ô.

5.2.6 Tạo giao diện bàn chơi (Grid Buttons)

```
1 def run_game(difficulty):
2     game = Minesweeper(difficulty)
3
4     button_layout = []
5     for r in range(game.rows):
6         row_list = []
7         for c in range(game.cols):
8             btn_key = f"{r},{c}"
9             btn = sg.Button("", size=(2, 1), key=btn_key, pad=(0, 0),
10                             font=("Helvetica", 10, "bold"),
11                             button_color=('white', 'gray'))
12             row_list.append(btn)
13         button_layout.append(row_list)
```

Listing 8: Tạo lưới nút đại diện cho bàn cờ

Sau khi tạo đối tượng logic `game`, chương trình xây dựng lưới giao diện. Mỗi ô `(r,c)` tương ứng một nút `Button` và có khóa `key="r,c"`. Khóa này giúp map trực tiếp từ sự kiện giao diện về đúng vị trí trong `game.board`.

```
1 layout = [
2     [sg.Text(f"Mines: {game.mines}", key='INFO', font=("Helvetica", 12))],
3     [sg.Column(button_layout)]
4 ]
5 window = sg.Window("Minesweeper", layout, finalize=True)
```

Listing 9: Tạo cửa sổ game và hiển thị số mìn

Giao diện gồm một dòng hiển thị số mìn và một `Column` chứa toàn bộ lưới nút. Tham số `finalize=True` cho phép truy cập và bind sự kiện lên các phần tử sau khi tạo cửa sổ.

5.2.7 Gán sự kiện chuột phải

```
1 for r in range(game.rows):
2     for c in range(game.cols):
3         window[f"{r},{c}"].bind('<Button-3', '+RIGHT')
```

Listing 10: Bind chuột phải cho từng ô

`FreeSimpleGUI` không có callback chuột phải mặc định, nên chương trình dùng `bind` của `Tkinter`: `<Button-3` là chuột phải. Chuỗi hậu tố `+RIGHT` giúp phân biệt click phải với click trái khi đọc event.

5.2.8 Vòng lặp sự kiện và phân tích event

```
1 while True:
2     event, _ = window.read()
3
4     if event == sg.WIN_CLOSED:
5         window.close()
6         return False
```

Listing 11: Vòng lặp đọc sự kiện

`window.read()` chờ sự kiện. Nếu người dùng đóng cửa sổ, hàm trả về `WIN_CLOSED`. Chương trình đóng cửa sổ và trả về `False` để vòng lặp chính kết thúc.

```
1 r, c = -1, -1
2 is_right_click = False
3
4 try:
5     if '+RIGHT' in event:
6         coords = event.replace('+RIGHT', '')
7         r, c = map(int, coords.split(','))
8         is_right_click = True
9     elif ',' in event:
10        r, c = map(int, event.split(','))
11    else:
12        continue
13 except ValueError:
14    continue
```

Listing 12: Giải mã tọa độ và loại click từ event

Biến `event` là chuỗi. Nếu event chứa `+RIGHT` nghĩa là click phải, chương trình tách chuỗi để lấy tọa độ. Nếu chỉ có dạng `"r,c"` thì là click trái. Khối `try/except` giúp tránh crash khi gặp event không đúng định dạng.

5.2.9 Gọi logic: đặt cờ và mở ô

```
1 if is_right_click:
2     game.toggle_flag(r, c)
3 else:
4     game.reveal(r, c)
5
6 update_board(window, game)
```

Listing 13: Gọi hàm xử lý logic theo thao tác

Click phải gọi `toggle_flag` để bật/tắt cờ tại ô. Click trái gọi `reveal` để mở ô. Sau đó chương trình gọi `update_board` để cập nhật giao diện theo trạng thái mới.

5.2.10 Logic mở ô (Reveal) và trạng thái thua

```
1 def reveal(self, r, c):
2     if not (0 <= r < self.rows and 0 <= c < self.cols):
3         return
4     cell = self.board[r][c]
5     if cell['open'] or cell['flag']:
6         return
7
8     cell['open'] = True
9
10    if cell['mine']:
11        self.game_over = True
```

```
12 return
```

Listing 14: Mở ô và xử lý nếu gặp mìn

Hàm `reveal` kiểm tra biên, tránh mở lại ô đã mở hoặc ô đang bị gắn cờ. Khi mở ô, nếu ô đó là mìn (`mine=True`) thì đặt `game_over=True` để kết thúc game.

5.2.11 Flood Fill khi ô trống (`neighbors = 0`)

```
1 if cell['neighbors'] == 0:
2     self._flood_fill(r, c)
3
4 self._check_victory()
```

Listing 15: Kích hoạt Flood Fill

Nếu ô vừa mở không có mìn kề, chương trình tự động mở các ô an toàn liên tiếp bằng flood fill. Sau khi xử lý xong, chương trình kiểm tra điều kiện thắng.

```
1 def _flood_fill(self, r, c):
2     stack = [(r, c)]
3     visited = set()
4     while stack:
5         curr_r, curr_c = stack.pop()
6         if (curr_r, curr_c) in visited:
7             continue
8         visited.add((curr_r, curr_c))
9
10        directions = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
11
12        for dr, dc in directions:
13            nr, nc = curr_r + dr, curr_c + dc
14            if 0 <= nr < self.rows and 0 <= nc < self.cols:
15                neighbor = self.board[nr][nc]
16                if not neighbor['open'] and not neighbor['flag'] and not neighbor['mine']:
17                    neighbor['open'] = True
18                    if neighbor['neighbors'] == 0:
19                        stack.append((nr, nc))
```

Listing 16: Thuật toán Flood Fill dùng stack

Flood fill được cài đặt theo kiểu duyệt đồ thị với `stack` để tránh đệ quy sâu. Mỗi ô trống sẽ mở các ô xung quanh; nếu ô xung quanh cũng trống (`neighbors=0`) thì tiếp tục đưa vào stack để lan rộng vùng an toàn.

5.2.12 Đặt/Gỡ cờ (Flag)

```
1 def toggle_flag(self, r, c):
2     if 0 <= r < self.rows and 0 <= c < self.cols:
3         cell = self.board[r][c]
4         if not cell['open']:
5             cell['flag'] = not cell['flag']
```

Listing 17: Bật/tắt cờ cho ô chưa mở

Người chơi chỉ được đặt cờ trên ô chưa mở. Mỗi lần click phải, `flag` sẽ đảo trạng thái (đang không có cờ → có cờ và ngược lại).

5.2.13 Kiểm tra điều kiện thắng

```
1 def _check_victory(self):
2     safe_cells_closed = 0
3     for row in self.board:
4         for cell in row:
5             if not cell['mine'] and not cell['open']:
6                 safe_cells_closed += 1
7     if safe_cells_closed == 0:
8         self.victory = True
```

Listing 18: Chiến thắng khi mọi ô an toàn đều đã mở

Chương trình đếm số ô an toàn còn chưa mở. Nếu bằng 0, tức là mọi ô không mìn đã mở hết, đặt victory=True. Điều kiện này không phụ thuộc vào việc người chơi đặt cờ đúng hay sai.

5.2.14 Cập nhật giao diện theo trạng thái board

```
1 def update_board(window, game):
2     for r in range(game.rows):
3         for c in range(game.cols):
4             cell = game.board[r][c]
5             btn_key = f"{r},{c}"
6
7             if cell['open']:
8                 if cell['mine']:
9                     window[btn_key].update('*', button_color=('white', 'red'))
10                else:
11                    text = str(cell['neighbors']) if cell['neighbors'] > 0 else ""
12                    window[btn_key].update(text, button_color=('black', 'lightgray'), disabled=True)
13                elif cell['flag']:
14                    window[btn_key].update('P', button_color=('red', 'gray'))
15                else:
16                    window[btn_key].update('', button_color=('white', 'gray'))
```

Listing 19: Cập nhật hiển thị từng ô lên GUI

Hàm update_board đồng bộ logic với GUI:

- Nếu ô đã mở và là mìn: hiển thị * và tô đỏ.
- Nếu ô đã mở và an toàn: hiển thị số mìn kề (nếu có) và khóa nút (disabled=True).
- Nếu ô chưa mở nhưng có cờ: hiển thị P.
- Nếu ô chưa mở và không cờ: giữ trống.

5.2.15 Kết thúc trò chơi (Game Over / Victory)

```
1 if game.game_over:
2     sg.popup("BOOM! You lost.", title="Game Over")
3     break
4 if game.victory:
5     sg.popup("Congratulations! You won!", title="Victory")
6     break
```

Listing 20: Thông báo thua hoặc thắng



Sau mỗi lượt xử lý, chương trình kiểm tra hai cờ trạng thái: `game_over` khi mở trúng mìn và `victory` khi mở hết ô an toàn. Nếu xảy ra, chương trình hiển thị thông báo và thoát vòng lặp chơi.

6 Kết quả thực nghiệm (Experimental Results)

6.1 Môi trường cài đặt

Ứng dụng được phát triển và thử nghiệm trên môi trường máy cá nhân với cấu hình như sau:

- Hệ điều hành: Windows 10 64-bit
- Python: 3.13
- Thư viện giao diện: FreeSimpleGUI
- IDE sử dụng: Visual Studio Code

Ứng dụng không yêu cầu phần cứng đặc biệt và có thể chạy ổn định trên các máy tính phổ thông. Việc cài đặt được thực hiện thông qua file `requirements.txt` với lệnh:

```
pip install -r requirements.txt
```

Sau khi cài đặt thư viện, chương trình được chạy bằng lệnh:

```
python main.py
```

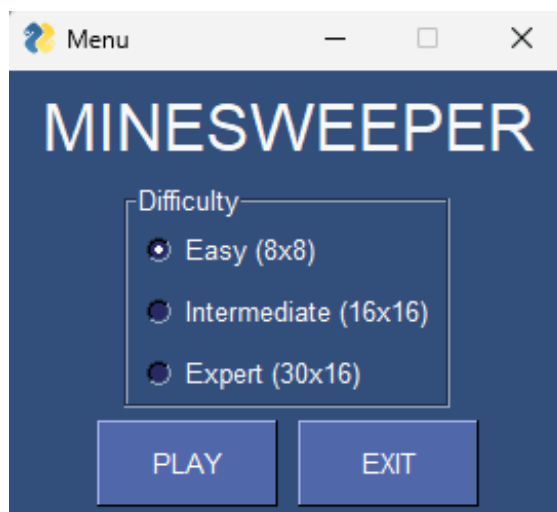
Thời gian khởi chạy ứng dụng dưới 1 giây và không phát sinh lỗi trong quá trình thực nghiệm.

6.2 Demo các trường hợp

Để đánh giá tính đúng đắn của chương trình, nhóm tiến hành thử nghiệm các trường hợp chính của trò chơi Minesweeper, bao gồm khởi động game, quá trình chơi, thua cuộc và chiến thắng.

1. Màn hình chính

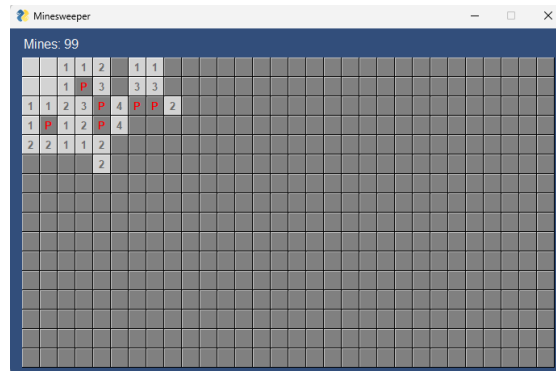
Khi khởi động, người chơi có thể chọn độ khó gồm Easy, Intermediate và Expert. Giao diện đơn giản, trực quan và dễ sử dụng.



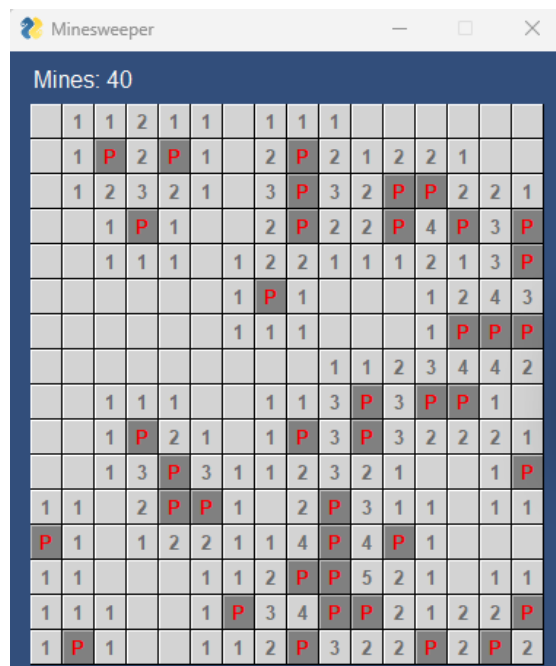
Hình 3: Màn hình menu chọn độ khó

2. Quá trình chơi

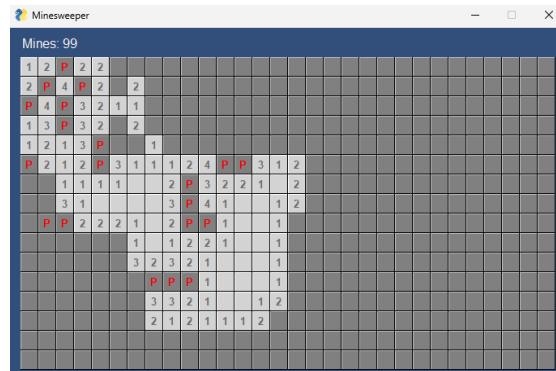
Sau khi chọn độ khó, bảng Minesweeper được tạo ngẫu nhiên. Người chơi có thể click trái để mở ô và click phải để đặt cờ đánh dấu mìn.



Hình 4: Trạng thái đang chơi



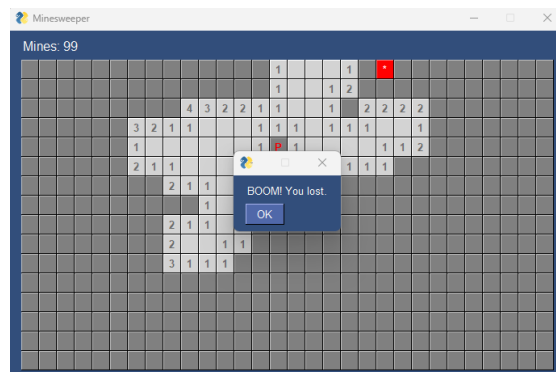
Hình 5: Trạng thái đang chơi



Hình 6: Trạng thái đang chơi

3. Trường hợp thua

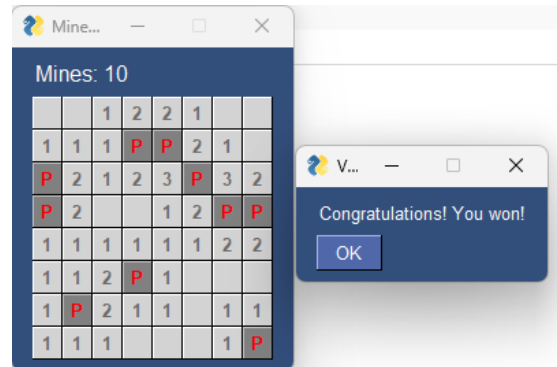
Khi người chơi mở phải ô chứa mìn, trò chơi kết thúc và hiển thị thông báo thua. Ô mìn được hiển thị màu đỏ để minh họa vị trí gây thua.



Hình 7: Trường hợp người chơi thua khi mở trúng mìn

4. Trường hợp chiến thắng

Khi tất cả các ô không chứa mìn được mở, chương trình hiển thị thông báo chiến thắng. Kết quả này xác nhận logic kiểm tra thắng hoạt động chính xác.



Hình 8: Thông báo chiến thắng

Qua các thử nghiệm trên, chương trình hoạt động ổn định, không xảy ra lỗi trong quá trình chơi và đáp ứng đúng các luật cơ bản của trò chơi Minesweeper.

7 Kết luận (Conclusion)

7.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và hiện thực, nhóm đã hoàn thành các mục tiêu đề ra của bài tập lớn. Các kết quả cụ thể bao gồm:

- **Sản phẩm hoàn thiện:** Một ứng dụng Minesweeper hoạt động ổn định trên nền tảng Python 3.13, sử dụng thư viện `FreeSimpleGUI` cho giao diện thân thiện và dễ sử dụng.
- **Cài đặt thành công thuật toán DFS:** Nhóm đã áp dụng thành công thuật toán Tìm kiếm theo chiều sâu (DFS) sử dụng ngăn xếp (Stack) để xử lý cơ chế loang (Flood Fill). Thuật toán hoạt động chính xác, hiệu quả, giúp mở rộng vùng an toàn nhanh chóng mà không gặp lỗi tràn bộ nhớ (Stack Overflow) như đệ quy thông thường.
- **Kiến trúc chương trình rõ ràng:** Mã nguồn được tổ chức khoa học, tách biệt rõ ràng giữa lớp xử lý logic (`minesweeper.py`) và lớp giao diện (`main.py`), tuân thủ nguyên lý lập trình hướng đối tượng.
- **Đầy đủ tính năng:** Trò chơi hỗ trợ 3 mức độ khó (Easy, Intermediate, Expert), chức năng cấm cờ, nhận diện thắng/thua chính xác và cho phép chơi lại nhiều lần.

7.2 Hạn chế và Hướng phát triển

Mặc dù đã đáp ứng các yêu cầu cơ bản, sản phẩm vẫn còn một số hạn chế và có thể được cải tiến thêm trong tương lai:

Hạn chế:

- **Giao diện:** Mặc dù `FreeSimpleGUI` gọn nhẹ nhưng giao diện chưa thực sự bắt mắt và hiện đại so với các game thương mại. Hiệu ứng chuyển động (animation) khi mở ô còn hạn chế.



- **Đảm bảo lượt đầu tiên:** Thuật toán rải mìn hiện tại là ngẫu nhiên hoàn toàn ngay từ khi khởi tạo. Do đó, vẫn có xác suất nhỏ người chơi click trúng mìn ngay lượt đầu tiên (Instant Death). Các phiên bản Minesweeper hiện đại thường dời mìn đi nếu người chơi click trúng vào lượt đầu.

Hướng phát triển:

- **Tích hợp AI Agent:** Đây là hướng phát triển quan trọng nhất, phù hợp với môn học Trí tuệ Nhân tạo. Nhóm có thể phát triển thêm một Agent sử dụng Logic mệnh đề (Propositional Logic) hoặc CSP (Constraint Satisfaction Problem) để tự động giải game thay vì chỉ để con người chơi.
- **Cải thiện UX/UI:** Nâng cấp giao diện đẹp mắt hơn, thêm bảng xếp hạng (Leaderboard) lưu thành tích người chơi.
- **Tối ưu hóa thuật toán:** Cải thiện thuật toán rải mìn để đảm bảo tính công bằng và luôn giải được (solvable) mà không cần đoán mò ở những nước cuối.

Tài liệu tham khảo

- [1] Python Documentation, <https://docs.python.org/3/>
- [2] FreeSimpleGUI Documentation, <https://github.com/spyoungtech/FreeSimpleGUI>
- [3] Introduction to Artificial Intelligence Lecture Notes, HCMUT.